

Cheat Sheet 1/4 · 编程基础 (Ch 01–05)

01 编程语言概述

- **冯·诺依曼**: CPU + 内存 + 总线 + I/O
- **存储程序原理**: 代码与数据在内存中等价
- **编译 vs 解释**:
 - 编译: 源码 → AST → 字节码 / 机器码
 - 解释: 直接遍历 AST 求值
- **编译器流程**:
词法 → 语法 → 语义 → IR → 优化 → 目标

02 可计算性

- **Church–Turing 论题**:
图灵机 ≡ Lambda 演算 ≡ μ -递归
- **停机问题不可解** (对角线法证明)
- **图灵完备**: 能模拟任意图灵机

03 Lambda 演算

- 三件套: **变量 / 函数抽象 / 函数应用**
- **β -归约**: $(\lambda x. E) V \rightarrow E[x:=V]$
- **α -等价**: 变量重命名不改变语义
- **η -规约**: $\lambda x. f(x) \equiv f$
- **求值策略**:
 - CBV (应用序) — Java、Scheme
 - CBN (正则序) — Haskell
- **Church 编码**: 用纯函数表达数字 / 布尔

04 程序的结构

- **代码是用来读的** — 命名 > 注释
- **降低复杂度三剑客**:
 - 分解 (Decomposition)
 - 抽象 (Abstraction)
 - 层次性 (Hierarchy)
- **Wirth 公式**: `Program = Algorithm + DS`
- **三种语言机制**: 基本表达式 · 组合 · 抽象

05 函数式范式

- **避免重复**的 5 个层次:
 - 复制 → 循环 → 抽函数 → 数据驱动 → 高阶函数
- **纯函数**: 相同输入 → 相同输出, 无副作用
- **不可变 + 引用透明性**:
 - 表达式可被其值替换
 - 可推理 / 可缓存 / 可并发
- **函数 = 一等公民**: 可作参数 / 返回值 / 字段

三层之间的关系

理论底座 (01–05): 什么是计算?
↓ 暴露问题
结构化 (06–09): 怎么写代码?
↓ 暴露问题
面向对象 (10–12): 怎么写得能维护?

Lab 01–08 的 Lambda 解释器是理论的实现物证。

Cheat Sheet 2/4 · 结构化 + OOP 概念

06 结构化思想 (Ch 06)

- 自顶向下分解 · 模块化 · 信息隐藏
- **痛点**: 数据 + 操作分离 → 维护性差

07 变量与数据类型 (Ch 07)

- **栈 vs 堆**: 变量在栈, 对象在堆
- **赋值的代价** (SICP 3.1):
 - 打破代换模型 → 引入环境模型
 - 打破引用透明性
 - 引入顺序依赖、同一 vs 等价

08 控制流与方法 (Ch 08)

- **三大基本结构** (Bohm–Jacopini): 顺序 · 分支 · 循环
- **递归 vs 迭代**: TCO 把尾递归编译成跳转
- 函数 = 输入 → 输出 + 前/后置条件

09 软件工程建模 (Ch 09)

图	视角
用例图	用户视角的功能
类图	静态结构
顺序图	动态行为 (时序)
通信图	对象间链接 (拓扑)

用例 ↔ 类图 ↔ 顺序图 互补、互验。

10 OOP 思想 (Ch 10)

- 软件 = 一组对象间的协作
- **职责** = 数据职责 + 行为职责
- **三大维护性问题**:
 - 实现变更 · 新增需求 · 格式变更
- **三大原则**: 封装 · 继承 · 多态

11 OOP 封装 (Ch 11)

- `private` 字段 + `public` 方法 = 受控暴露
- `static` (类) vs 实例字段 / 方法
- `final`: 类不可继承 / 方法不可覆盖 / 字段不可改
- `==` vs `equals` / `hashCode` 必须**成对**重写
- **GC**: 可达性分析 · 三件任务

12 OOP 协作 (Ch 12)

- **顺序图**表达动态协作
- **Demeter 法则**: 只与最直接朋友交流
- 关系强度:
依赖 < 关联 < 聚合 < 组合
- **协作模式**:
Client / Server / Manager / Delegate
- **DI / IoC**: Manager 模式的工业化 (Spring)

Cheat Sheet 3/4 · Java 语法核心卡

类骨架

```
public class Foo extends Bar
    implements Comparable<Foo> {
    private final int value;           // 字段
    public Foo() { this(0); }          // this 链
    public Foo(int v) {
        super("foo");                 // super 第一行
        this.value = v;
    }
    @Override public int compareTo(Foo o)
    { return Integer.compare(value, o.value); }
    public static final int MAX = 100;
}
```

equals 五步模板

```
public boolean equals(Object o) {
    if (this == o) return true;       // ① 自反
    if (!(o instanceof Foo)) return false; // ② 类型
    Foo f = (Foo) o;                  // ③ 强转
    return Objects.equals(value, f.value); // ④ 字段
}                                     // ⑤ 配 hashCode
```

== vs equals 三铁律

- 1. 基本类型用 ==
- 2. 引用类型用 equals()
- 3. 字面量字符串可 == (常量池),
new String("..") 不行

构造方法 + 静态初始化

- this(...) 必须第一行；不能循环调用
- this(...) 与 super(...) 二选一
- 初始化顺序：
 - i. static 字段 + static 块 (按代码顺序)
 - ii. 实例字段 + 实例块
 - iii. 构造方法

包装类陷阱

```
Integer a = 127, b = 127; // a == b → true (缓存)
Integer c = 128, d = 128; // c == d → false (新对象)
```

包装类比较永远用 equals()。

Object 重写

方法	何时重写
equals + hashCode	内容比较时 (成对)
toString	调试 / 日志
compareTo	想排序时 (Comparable)

函数式 API (Java 8+)

```
stream().filter(x -> ...).map(...).collect()
Optional.ofNullable(x).orElse(default)
String::length           ← 方法引用
(x, y) -> x + y          ← Lambda 表达式
```

OOP 设计 8 问

- 1. **SRP**: 只有一个变化原因吗？
- 2. **OCP**: 新增功能能否不改老代码？
- 3. **LSP**: 父类的位置都能换成子类吗？
- 4. **DIP**: 依赖抽象而非具体？
- 5. **Demeter**: 是否火车残骸 `a.b().c().d()` ？
- 6. **关联**: 用 / 持有 / 拥有 / 共生死？
- 7. **继承 vs 组合**: 默认优先组合
- 8. **封装**: 字段都 `private` 吗？

关联关系判定（语义）

短语	关系	例子
<code>... uses a ...</code>	依赖	临时变量
<code>... has a ...</code>	关联	长期持有
<code>... owns a ...</code>	聚合	共享 ownership
<code>... is part of</code>	组合	共生死

继承 vs 组合（默认组合）

问句	是	否
<code>A is a B ?</code>	继承 ✓	组合
<code>A has a B ?</code>	组合 ✓	继承

字段隐藏 vs 方法覆盖

```
P p = new C();
p.method()    // 看实际类型（动态）
p.field       // 看声明类型（静态！）
```

字段不参与多态，方法才参与。

高频易错速查（30 秒）

现象	怎么改
<code>new Integer(1) == new Integer(1)</code> 是 false	用 <code>equals</code>
<code>HashMap</code> 找不到对象	<code>equals</code> + <code>hashCode</code> 成对重写
<code>final List l</code> 还能 <code>add</code>	用 <code>unmodifiableList</code>
<code>Counter</code> 被外部改了	字段 <code>private</code> + <code>getter</code>
子类构造方法报错	显式 <code>super(...)</code>
用 <code>==</code> 比字符串失败	用 <code>equals</code>

字符串常量池速记

```
"abc" + "def"      // 编译期算 → 进池
"ab" + c + "ef"    // 运行期算 → 不进池
str.intern()       // 手动入池
```

Lambda 解释器知识地图

Lab	类 / 方法	章节根源
01	<code>CharLookaheadStream</code>	Ch 01
02	<code>Lexer</code> · <code>Token</code> 子类	Ch 01
03	<code>Parser</code> · <code>LObject</code>	Ch 01
04	<code>Evaluator</code> · <code>LFunction</code>	Ch 03
05	<code>Environment</code> · 作用域链	Ch 03
06	<code>fn*</code> / <code>if</code> / <code>do</code>	Ch 02
07	<code>LFunctionTco</code> · <code>while</code>	Ch 08
08	<code>LAtom</code> · <code>read-string</code>	Ch 02+03

使用建议：考前 30 分钟通读四页，红笔在最容易忘的条目旁画 ★，进考场前再扫 ★ 处。